[MIDWAY REPORT] IS THE GAP AN INTERFACE GAP? AGENT TOOL INTERFACES FOR DATA CATALOGS

Alex Feng

1 PROBLEM STATEMENT (PROBLEM SPECIFICATION)

Maintainers of lakehouses need catalog metadata (ownership, grain, filters, lifecycle state) that stays trustworthy as models evolve, but it drifts from the code—annotations "can become outdated or incorrect" as data models evolve (Lakafosis et al., 2025)—and the evidence for a correction is scattered across catalog metadata, SQL, lineage, tests, and the data. We are building a *catalog steward* for them with two triggers over one workflow: *remediation* of existing hygiene debt and *change review* of a proposed contribution. Our question: **does a specialized evidence-gathering, risk-aware action, and verification workflow produce more complete, grounded, and safe maintenance outcomes than a general-purpose coding agent with access to the same repository and warehouse?**

2 METHODOLOGY

2.1 ENVIRONMENT & DATA

Environment and data. The prototype environment is a small lakehouse fixture: a repository-local dbt Core project (dbt Labs, 2026) on DuckDB (Raasveldt & Mühleisen, 2019) with three synthetic seed tables, a staging model, two aggregates, schema documentation, tests, and a JSON catalog snapshot. It is a reproducible test bed, not evidence about production warehouses. A versioned hygiene contract (`design/hygiene_contract.yaml`) bounds the task to five issue types (missing required metadata, SQL/metadata drift, grain or join drift, unsupported reliability claims, lifecycle drift) and three action classes (patch, propose, report) under hard scope limits. The pilot suite (`data/episodes/manifest.yaml`) has ten hand-built episodes: four *remediation* cases, four *flawed changes*, and two *clean controls*. Each records its expected `(issue_type, asset)` findings and its acceptable and unsafe actions.

Evaluation. Each condition runs on an identical, freshly built copy of each episode. Its findings are reduced to `(issue_type, asset)` pairs and matched exactly against the manifest, giving true positives (TP), extra findings (FP) and missed findings (FN). An episode succeeds only with no misses or extras, no disallowed action class and no recorded verification failure, and every saved trace is then audited. Appendix A gives the fixture, contract and episodes in full, and the step-by-step procedure with what the scorer does not check.

2.2 METHODS

What we built. The catalog steward (Figure 1) pairs a domain-specific evidence interface for catalog hygiene with an inspect–plan–act–verify workflow under a risk-aware action policy, and both triggers share it: remediation starts from existing hygiene debt, and change review adds the proposed diff. This pairing is the contribution. Rather than leaving a model to search a repository and edit files, the steward first gathers the facts that hygiene decisions rest on into an evidence bundle (catalog metadata, dbt schema and manifest, model SQL with facts parsed by SQLGlot (Mao, 2026), lineage, bounded data profiles), each item carrying a stable, content-hashed ID. Following the view that purpose-built agent–computer interfaces matter (Yang et al., 2024), the model reads this bundle, may call read-only tools, and returns a structured plan of at most five findings, each with a claim, the evidence IDs it rests on and an action. A deterministic validator rejects a plan that breaks the schema or the contract's scope, evidence or action rules, with one repair turn. It then routes each finding to a catalog patch, a review proposal or a report, and appends every outcome to a ledger. Only reversible technical metadata may be patched, and only in an episode-local override file.

Limits. Verification is weak. A patch is kept only if the written field equals the value the model declared and `dbt parse` still succeeds. Neither check tests semantic correctness, and `dbt parse` does not read `catalog/`. The contract's per-issue checks (dashed in the figure) are not implemented yet. Appendix B details the evidence, tools, validator rules and verification.

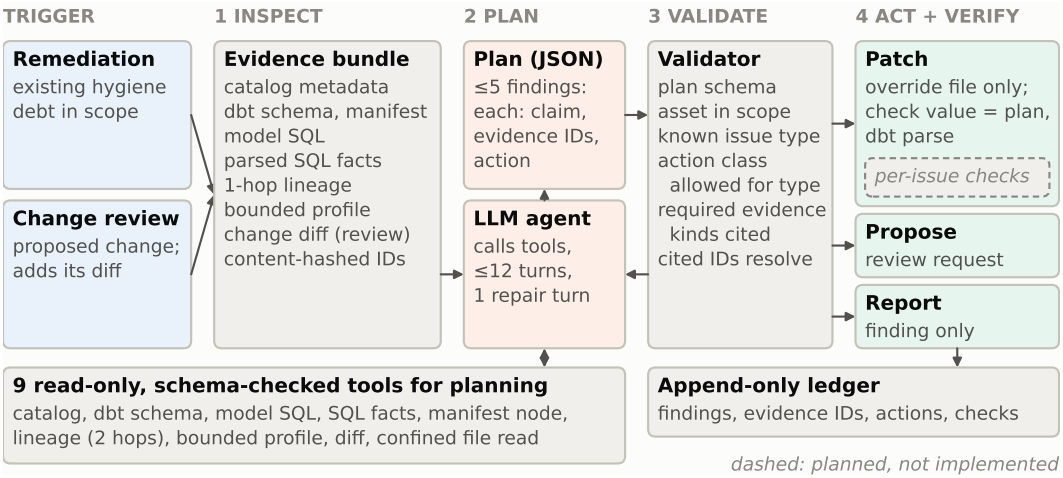


Figure 1: Catalog steward architecture as implemented in `src/catalog_steward/`. Both triggers feed one workflow. The model reads a bundle of content-hashed evidence, may call read-only tools, and returns a structured plan. A deterministic validator accepts the plan or returns it once for repair, and each accepted finding is then patched, proposed or reported, and logged. The only planned element is dashed: the contract's per-issue verification checks.

Conditions. The steward uses `openai/gpt-6-luna` (low reasoning effort, at most 12 turns) under frozen configuration `experiments/configs/midpoint_v1.yaml` v1.2.0. For two selected cases we also ran OpenAI Codex CLI 0.167.0 (OpenAI) with `gpt-6.1-terra` at medium reasoning effort on separately built fixtures. Codex had general repository tools and could edit files; the steward received a specialized evidence bundle and operated through a validated plan and patch executor. Their models, prompts, tools, limits and execution differ, so this comparison cannot isolate an interface effect. The original pilot's custom raw-tool workflow is not Codex and is excluded from this comparison.

2.3 What Did Not Work, and Pivots

From catalog questions to maintenance. The proposal's large catalog-question benchmark was narrowed after feedback to two maintenance triggers and a local fixture. The system now produces evidence-linked actions, but the pilot does not establish a production safety improvement.

Protocol errors. In an earlier two-episode run the model copied a reply wrapper instead of the expected object, causing rejected replies. The team clarified the shared format example and added one repair turn before the pilot. A later pilot trace for `change_daily_revenue_filter` shows the exploratory agent batching tool calls into a rejected reply and returning no finding; it missed the expected regression. The team did not tune the protocol after inspecting this pilot. The saved `change_daily_revenue_filter/generic.json` and pilot `analysis.md` record the outcome.

3 Results (Summary of Progress)

Completed work and matched cases. The fixture, evidence tools, steward workflow and evaluator are implemented. Table 1 compares the two selected cases using exact manifest labels. TP is a matching finding; FP is an extra finding; FN is a missed expected finding. Success requires no extras or misses and the checks described above. Each case has one record per condition; the revenue steward record reuses an earlier response. Codex findings were manually mapped from

its path-based output to the same labels and its actions inspected separately; this was an adaptation of the scorer, not a run of the steward executor. Prompts, inputs, traces and scoring are in `experiments/results/codex_midway_two_case/README.md`.

Table 1: Two illustrative episodes, one run per condition; matches and extras are relative to the existing manifest.

Case	Steward TP/FP/FN	Codex TP/FP/FN	Steward / Codex success
Revenue-filter repair	1/0/0	1/0/0	yes / yes
Changed grain	2/2/0	2/2/0	no / no

Case 1: revenue-filter repair. In `rem_daily_revenue_filter_drift`, the catalog says all orders while SQL filters to completed orders. The expected finding is SQL/metadata drift on `daily_revenue`. The steward cited evidence and patched the description to completed orders: 1 TP, 0 FP, 0 FN, and episode success. Codex also patched the override and compared the JSON description; it did not run dbt tests. The steward checked written equality and dbt parsing, not semantic correctness. The pilot folder contains the plan and ledger for this episode.

Case 2: changed grain. In `change_customer_orders_grain`, the manifest expects grain drift and an unsupported reliability claim on `customer_orders`. The steward found both, but also reported a duplicate SQL-drift issue and an undocumented seed: 2 TP, 2 FP, 0 FN, so the episode fails despite complete recall. Codex also reported both expected issues and the same two extras, requested review, and made no changes. It calculated join cardinalities from CSV rather than running dbt tests. The seed is a real pre-existing gap, illustrating a disagreement between scope/labels and findings rather than an invented problem. The pilot folder contains this episode's plan and the audit.

Limitations. This is a small, single-sample, AI-constructed suite; both clean controls contain unlabelled gaps, and four records in the original pilot reuse earlier responses. The validator's issue-type rules can be bypassed by a mislabel: on the wrong-grain case the steward patched a correct value even though that episode accepts only a proposal with uniqueness evidence. Verification does not check that semantic postcondition. These two cases show no observed difference in exact-label scores, not evidence of equivalence or general superiority. Independent label review, more cases, repeated runs and matched ablations remain future work.

4 Assets

Repository: `https://github.com/alfenn/agentic-catalog-interface` (Feng, 2026): `README.md` (setup, reproduction), `src/catalog_steward/`, `data/` (fixture, episodes), and `experiments/results/midpoint_v1_2_pilot/` (pilot results, audit `analysis.md`). Midpoint demo video: **[VIDEO URL TO BE ADDED]**.

5 Timeline (Summary of Progress: Next Steps and Risks)

Milestones. Course deadlines are Demo Day on December 9 and the final 8-page report on December 11; intermediate dates are our targets.

- *By November 13:* a new frozen episode version: documented seeds, truly clean controls, an issue-type precedence rule checked independently of the model's label, and human review of the AI-built labels.
- *By November 20:* postcondition-based success and episode-level unsafe-action checks; real per-issue patch verification; a batching-robust protocol.
- *By December 4:* a matched coding-agent baseline, tool-versus-workflow ablations, and growth toward 30–50 audited episodes.
- *December 9 and 11:* live demo of both modes; final report.

Obstacles and concerns. No credential or pipeline blocker remains. The main concern is validity: label choices and protocol behaviour move results as much as the workflow, so fixes go into a new frozen version, not a rescoring.

Risks and mitigations. *Scope growth:* a frozen five-type contract with hard limits. *Ambiguous ground truth* (seen in both clean controls): documented seeds, a precedence rule, human label review. *Unsupported claims* (one invented-evidence run passed): claim–evidence support checks and manual audit. *Unmatched comparison:* align model, prompts and action limits, then test bundle ablations. *Warning overload* (six steward extras): clean controls, a five-finding cap, flagging only gaps a change introduces. *Small pilot:* per-episode reporting, more episodes.

REFERENCES

dbt Labs. dbt Core, version 1.12.6. GitHub repository `dbt-labs/dbt-core` (Apache-2.0), 2026. URL `https://github.com/dbt-labs/dbt-core`.

Alex Feng. agentic-catalog-interface: Catalog steward midpoint prototype, hygiene contract, episode manifest, evaluation configuration, and pilot results. GitHub repository; paths in the text are relative to its root, 2026. URL `https://github.com/alfenn/agentic-catalog-interface`.

Vasileios Lakafosis, Hannes Roth, Benjamin Renard, Wenlong Dong, Zhonghu Gao, and Dave Kurtzberg. How Meta understands data at scale. Engineering at Meta Blog, April 28, 2025, 2025. URL `https://engineering.fb.com/2025/04/28/security/how-meta-understands-data-at-scale/`.

Toby Mao. SQLGlot: An easily customizable SQL parser and transpiler, version 29.0.1. GitHub repository `tobymao/sqlglot` (MIT), 2026. URL `https://github.com/tobymao/sqlglot`.

OpenAI. Codex CLI. GitHub repository. URL `https://github.com/openai/codex`. Accessed November 8, 2026.

Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, 2019.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

A ENVIRONMENT, EPISODES AND EVALUATION

Fixture. Three seed tables (5 customers, 8 orders, 10 order items) feed a staging model and two aggregates (`customer_orders`, `daily_revenue`) through the dbt-duckdb adapter, with schema documentation, dbt tests and a JSON catalog snapshot. A compact `catalog/overrides.json` sidecar injects controlled metadata changes. Each episode copies the clean base, applies a debt or change overlay, generates the diff, and builds the project (`dbt seed`, `run`, `parse`), so every condition starts from the same state.

Hygiene contract. The contract (v0.1.0, validated by a JSON Schema) names, for each of the five issue types, its required evidence, allowed and forbidden action classes, and intended verification. Limits: 20 assets, two lineage hops, five findings and 10,000 probed rows. Deleting assets, changing access policies, mutating a live catalog, inventing business semantics and unbounded queries are forbidden.

Episodes and ground truth. The four remediation cases are a stale revenue-filter description, wrong customer-order grain, a missing owner, and a legacy copy still marked active and canonical. The four flawed changes are a changed revenue filter, a changed grain without updated grain metadata or tests, a new undocumented model, and a rename without a lifecycle transition. The two clean controls are a documented new model and a documented revenue metric. For each episode the manifest records the expected `(issue_type, asset)` findings, required evidence, *sets* of acceptable and unsafe

actions, postconditions, and intended checks. Action sets let an escalation count as correct where business meaning is uncertain. In the filter-drift case, for example, proposing a description fix is acceptable but editing the SQL to match stale prose is unsafe. The clean controls are meant to measure spurious findings, although both contain unlabelled gaps.

Evaluation procedure. For each episode: (1) copy the base fixture; (2) apply the specified debt or change overlay and build the dbt project; (3) run each condition independently on that state with its recorded prompt and settings; (4) save the findings and normalize them to `(issue_type, asset)`, keeping evidence and actions; (5) match those pairs exactly to the manifest and count TP, FP and FN; (6) count the episode as successful only if it completes without missing or extra findings, disallowed action classes, or a recorded verification failure; and (7) inspect the saved trace. Precision is TP/(TP+FP) and recall is TP/(TP+FN); micro scores aggregate counts before dividing. The scorer checks that evidence IDs resolve, not that they support the claim. It checks action classes, not episode-specific acceptable actions or postconditions. The AI-assisted trace audit examines those distinctions separately.

B System Details

Evidence bundle. For each scoped asset (at most 20) the steward collects catalog metadata (the snapshot merged with the override file), dbt schema documentation and tests, model SQL, parsed SQL facts (sources, filters, joins, aggregates, grouping, grain candidates, select expressions), the dbt manifest node, one-hop lineage, and a bounded profile. Change review adds the diff. The profile reads at most 10,000 rows from a read-only DuckDB connection with external access disabled and reports per-column null, distinct and duplicate counts. Every item has an ID derived from a hash of its kind, asset, source and content, plus a provenance pointer. Absent metadata is recorded as explicit `present: false` evidence. The prompt contains only the trigger, asset scope, evidence and collection errors, never the episode ID or labels.

Agent loop and tools. The model works through a JSON reply protocol for at most 12 turns. Three consecutive malformed replies end the run. During planning it may call nine read-only tools, whose arguments are checked against JSON Schemas: catalog metadata, dbt schema, model SQL, SQL facts, manifest node, lineage (up to two hops), a bounded profile of chosen columns, the change diff, and a file read confined to the episode project (at most 50 kB). No write tools are registered.

Validator and action policy. The final plan must be JSON that matches the steward plan schema, with at most five findings. Each finding must name an in-scope asset and an issue type in the contract, choose an action class that the contract allows for that issue type, cite only evidence IDs from this run, and cite every evidence kind the issue type requires. A failing plan is returned once with the error; a second failure ends the run with no action. Only missing required metadata and SQL/metadata drift allow a patch; grain, reliability and lifecycle issues allow only a proposal or a report. Because these rules key on the model's own issue label, a mislabelled finding can bypass them.

Execution, verification and ledger. A patch may set one of five fields (description, owner, lifecycle, grain, canonical) in the episode's `catalog/overrides.json`. The steward saves a unified diff and writes the file. It then re-reads the merged catalog metadata to check that the field equals the value the plan declared, and runs `dbt parse`. If either check fails, it restores the file and records a verification failure. A proposal or report is checked only for an allowed action class and resolved evidence IDs. Each outcome, with its finding, checks and patch path, is appended to `steward_ledger.jsonl` alongside a run summary. A finding already recorded as verified is not executed again. The contract's per-issue verification checks (for example, comparing metadata with parsed SQL, or rerunning a uniqueness probe) are not implemented.

C AI-Use Disclosure Statement

Generative AI agents did most of the implementation and writing at this stage. Codex also ran the two-case baseline and revised this report in response to human feedback. OpenAI Codex agents implemented the prototype (environment setup, agent harness, hygiene contract, evidence tools, steward workflow, and baseline runners), built the dbt/DuckDB fixtures and the ten evaluation episodes

and their labels, wrote the evaluation and scoring code, ran and audited the static pilot, and drafted the README and an initial version of this report. Claude (Anthropic) agents then audited, tested, fixed, and finished the work, including the preflight audit of the paid-run pipeline and its fixes, running the preliminary runs and the three-condition pilot, the manual audit of every pilot trace, typesetting this report in the ICLR template, the references, the script that generates the charts and results table from saved results, and the revision of this report around the audit. Because AI agents constructed the episodes and labels as well as the system under test, the ground truth reflects their design choices and needs independent human review. **[AUTHOR TO COMPLETE: describe the human verification actually performed, e.g., which code, episode labels, results, and report claims the author reviewed.]** AI tools are not authors; the author takes full responsibility for the content of this work.